

PERTEMUAN 5

STACK (LIFO – LAST IN FIRST OUT)

1. DESKRIPSI MODUL

Stack merupakan salah satu struktur data linear yang paling penting dalam ilmu komputer. Struktur data ini bekerja berdasarkan prinsip **Last In First Out (LIFO)**, yaitu elemen yang terakhir masuk ke dalam struktur akan menjadi elemen pertama yang keluar.

Konsep Stack banyak digunakan dalam berbagai aplikasi komputer modern seperti sistem operasi, compiler, browser web, editor teks, manajemen memori, pemanggilan fungsi (function call), recursive programming, hingga sistem navigasi aplikasi.

Pada modul ini mahasiswa akan mempelajari konsep dasar Stack, operasi-operasi utama yang terdapat pada Stack, implementasi menggunakan Python, analisis kompleksitas algoritma, serta penerapannya dalam berbagai studi kasus dunia nyata.

Setelah menyelesaikan praktikum ini mahasiswa diharapkan mampu memahami mekanisme kerja Stack dan mengimplementasikannya dalam penyelesaian berbagai permasalahan komputasi.

2. TUJUAN PRAKTIKUM

Setelah mengikuti praktikum ini mahasiswa diharapkan mampu:

1. Memahami konsep dasar Stack.
2. Menjelaskan prinsip kerja LIFO (Last In First Out).
3. Memahami operasi-operasi dasar Stack.
4. Mengimplementasikan Stack menggunakan Python.
5. Menganalisis kompleksitas operasi Stack.
6. Menerapkan Stack dalam penyelesaian masalah komputasi.
7. Mengembangkan program berbasis Stack.
8. Membandingkan Stack dengan struktur data linear lainnya.

3. CAPAIAN PEMBELAJARAN MODUL (CPM)

Mahasiswa mampu:

CPM 1

Menjelaskan konsep dan karakteristik Stack.

CPM 2

Mengimplementasikan operasi Push, Pop, Peek, IsEmpty, dan Size.

CPM 3

Menganalisis efisiensi algoritma pada operasi Stack.

CPM 4

Mengembangkan program berbasis Stack untuk penyelesaian masalah nyata.

CPM 5

Mengevaluasi penggunaan Stack dalam sistem komputer modern.

4. DASAR-DASAR TEORI

4.1 Pengertian Stack

Stack adalah struktur data linear yang bekerja berdasarkan prinsip:

Last In First Out (LIFO)

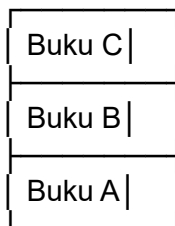
Artinya:

Data yang terakhir masuk akan menjadi data pertama yang keluar.

Contoh Kehidupan Sehari-hari

Tumpukan Buku

TOP



Jika ingin mengambil buku, maka Buku C harus diambil terlebih dahulu.

Tumpukan Piring

Piring yang terakhir diletakkan akan menjadi piring pertama yang diambil.

4.2 Karakteristik Stack

Karakteristik utama Stack:

- Bersifat linear
- Menggunakan prinsip LIFO
- Hanya memiliki satu titik akses (TOP)
- Operasi dilakukan pada elemen terbatas

4.3 Operasi Dasar Stack

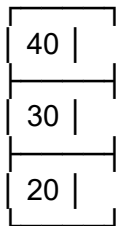
A. Push

Menambahkan elemen ke bagian atas Stack.

Visualisasi:

Push(40)

TOP

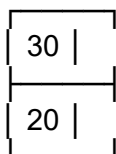


B. Pop

Menghapus elemen paling atas.

Pop()

TOP



C. Peek / Top

Melihat elemen paling atas tanpa menghapusnya.

D. IsEmpty

Memeriksa apakah Stack kosong.

E. Size

Menghitung jumlah elemen dalam Stack.

4.4 Representasi Stack Menggunakan Python

Stack dapat dibuat menggunakan List.

```
stack = []
```

Operasi Push

```
stack.append(10)
stack.append(20)
stack.append(30)
```

Operasi Pop

```
stack.pop()
```

Operasi Peek

```
print(stack[-1])
```

4.5 Kompleksitas Operasi Stack

Operasi	Kompleksitas
Push	$O(1)$
Pop	$O(1)$
Peek	$O(1)$
IsEmpty	$O(1)$
Size	$O(1)$

Karena semua operasi dilakukan pada elemen paling atas.

4.6 Stack dalam Sistem Komputer

Stack merupakan struktur data yang digunakan hampir di seluruh sistem komputer modern.

Contoh:

Function Call Stack

Ketika fungsi dipanggil:

```
def A():
    B()

def B():
    print("Hello")
```

Komputer menyimpan informasi fungsi pada Call Stack.

Visualisasi:

TOP

B()

A()

Main()

4.7 Stack dan Rekursi

Program rekursif menggunakan Stack secara otomatis.

Contoh:

```
def hitung(n):  
    if n == 0:  
        return  
    print(n)  
    hitung(n-1)
```

Setiap pemanggilan fungsi akan masuk ke Call Stack.

4.8 Stack Overflow

Stack Overflow terjadi ketika jumlah pemanggilan fungsi terlalu banyak sehingga memori Stack habis.

Contoh:

```
def loop():  
    loop()
```

loop()

Hasil:

RecursionError

4.9 Implementasi Dunia Nyata

Stack digunakan pada:

Browser History

- Back
- Forward

Undo dan Redo

Microsoft Word

Compiler

Pengecekan tanda kurung.

Sistem Navigasi

Android dan iOS.

Editor Kode

Undo perubahan kode.

5. ALAT DAN PERANGKAT

Perangkat Keras

- Laptop/PC
- RAM minimal 4 GB

Perangkat Lunak

- Python 3.x
- Visual Studio Code / PyCharm
- Google Colab
- Browser

6. PROSEDUR PRAKTIKUM

Praktikum 1 – Membuat Stack Sederhana

Langkah Kerja

1. Buka Python.
2. Buat list kosong.
3. Tambahkan beberapa data.
4. Tampilkan isi Stack.

Program

```
stack = []

stack.append(10)
stack.append(20)
stack.append(30)

print(stack)
```

Praktikum 2 – Operasi Pop

```
stack = [10,20,30]
data = stack.pop()
```

```
print("Data keluar =", data)
print(stack)
```

Praktikum 3 – Peek

```
stack = [10,20,30]
print(stack[-1])
```

Praktikum 4 – Simulasi Browser History

Mahasiswa membuat menu:

- Kunjungi halaman
- Back
- Tampilkan riwayat

Praktikum 5 – Validasi Kurung

Input:

`((a+b)*c)`

Output:

`Valid`

7. STUDI KASUS

Studi Kasus 1

Tumpukan Buku Perpustakaan

Buat simulasi penambahan dan pengambilan buku menggunakan Stack.

Studi Kasus 2

Browser History

Buat sistem riwayat halaman web yang mendukung fitur Back.

Studi Kasus 3

Undo Editor Teks

Buat program yang menyimpan riwayat perubahan dokumen.

Studi Kasus 4

Validasi Tanda Kurung

Periksa apakah pasangan tanda kurung pada ekspresi matematika valid.

Studi Kasus 5

Riwayat Transaksi ATM

Simpan transaksi terakhir nasabah menggunakan Stack.

8. TUGAS PRAKTIKUM

Level Mudah

Tugas 1

Buat program Stack sederhana menggunakan List.

Tugas 2

Implementasikan operasi Push dan Pop.

Tugas 3

Tampilkan elemen teratas menggunakan Peek.

Level Menengah

Tugas 4

Buat menu Stack interaktif.

Tugas 5

Hitung jumlah elemen Stack.

Tugas 6

Buat simulasi tumpukan piring.

Level Sulit

Tugas 7

Buat program validasi tanda kurung.

Tugas 8

Buat simulasi Browser History.

Tugas 9

Buat sistem Undo-Redo sederhana.

Tugas 10

Analisis kompleksitas setiap operasi Stack yang digunakan pada program Tugas 8 dan 9.

9. DISKUSI

1. Apa yang dimaksud dengan Stack?
2. Mengapa Stack menggunakan prinsip LIFO?
3. Apa perbedaan Push dan Pop?
4. Mengapa operasi Stack memiliki kompleksitas $O(1)$?
5. Apa fungsi Call Stack?
6. Apa hubungan Stack dengan Rekursi?
7. Apa yang menyebabkan Stack Overflow?
8. Mengapa fitur Undo menggunakan Stack?
9. Apa kelebihan dan kekurangan Stack?
10. Sebutkan minimal 5 penerapan Stack dalam kehidupan nyata dan sistem komputer.

10. FORMAT LAPORAN PRAKTIKUM

1. Halaman Judul
2. Tujuan Praktikum
3. Dasar Teori
4. Algoritma/Pseudocode
5. Program Python
6. Hasil Eksekusi Program
7. Screenshot Program
8. Analisis Hasil
9. Jawaban Diskusi
10. Kesimpulan
11. Daftar Pustaka

11. RUBRIK PENILAIAN

Komponen Penilaian	Bobot
Kehadiran	10%
Persiapan Praktikum	10%
Kode Program	25%
Ketepatan Output	20%
Analisis dan Pembahasan	15%
Laporan Praktikum	10%
Keaktifan Diskusi	10%
Total	100%